

# Unit 4

## Unit 4: Node.js, Express.js, MongoDB — Practical/Oral Exam Notes

### Context

Part of [Web Application Development](#) | Practical + Oral Exam Prep | SPPU Third Year IT

### High-Yield Oral Topics

- **Node.js Event Loop** — Non-blocking I/O explanation
- **Express Middleware** — What it is, how it works
- **REST API CRUD** — GET/POST/PUT/DELETE with routes
- **MongoDB vs SQL** — Always asked
- **Mongoose Schema** — How to define and use models
- **Full CRUD flow** — Frontend → API → DB → Response

## MINI PROJECT: Student Management System — Full Backend

Complete REST API: Node.js + Express + MongoDB for managing students.

## SECTION 1 — NODE.JS

### Exam Definition

Node.js is an **open-source, cross-platform JavaScript runtime environment** that executes JavaScript code **outside the browser**, built on Chrome's **V8 engine**. It uses an **event-driven, non-blocking I/O model** for high scalability.

### Non-Blocking I/O — Viva Explanation

"Traditional servers (Apache) create a new **thread** per request. If 1000 users hit the server, 1000 threads are created — expensive. Node.js uses **ONE thread** and the **Event Loop**. When a slow operation (DB query, file read) is requested, Node doesn't wait — it registers a callback and moves on. When the operation completes, the callback runs. This is **non-blocking I/O** and allows handling thousands of concurrent connections efficiently."

### SMS: Node.js Project Setup

Typical project setup involves:

- Initializing a project with `npm init` to create `package.json`.

- Installing external modules like Express and Mongoose via `npm install`.
- Configuring scripts for running the application.

## Node.js Built-in Modules

Core modules to know:

- `fs`: For reading and writing files.
- `http`: For creating a basic web server without frameworks.
- `path`: For handling and transforming file paths.
- `events`: For handling custom events via `EventEmitter`.

---

## SECTION 2 — EXPRESS.JS

### Exam Definition

Express.js is a **minimal, unopinionated web framework for Node.js** that provides routing, middleware support, and utilities for building web applications and REST APIs.

### Complete SMS REST API Server

An Express server typically consists of:

- **Initialization**: Creating the app instance and defining the port.
- **Middleware**: Parsing JSON, enabling CORS, and serving static files.
- **Database Connection**: Connecting to MongoDB using Mongoose.
- **Routing**: Mapping URL paths to specific route handlers.

## Middleware — Viva Critical Concept

Types of middleware:

- **Application-level**: Runs on all routes (e.g., body parser).
- **Route-level**: Runs only on specific routes (e.g., authentication checks).
- **Built-in**: Provided by Express (e.g., `express.static`).

**Middleware Flow**: Request → Logger MW → Auth MW → Route Handler → Response

---

## SECTION 3 — MONGODB + MONGOOSE

### Exam Definition

MongoDB is a **document-oriented NoSQL database** that stores data as **BSON documents** (Binary JSON) in collections instead of rows in tables.

# SQL vs NoSQL — MOST ASKED COMPARISON

| Feature        | SQL (MySQL)                 | NoSQL (MongoDB)                  |
|----------------|-----------------------------|----------------------------------|
| Data Format    | Tables with rows/columns    | Collections with JSON documents  |
| Schema         | Fixed, predefined           | Dynamic, flexible                |
| Relationships  | JOINS across tables         | Embedded documents or references |
| Scaling        | Vertical (bigger server)    | Horizontal (more servers)        |
| Query Language | SQL                         | MongoDB Query Language (MQL)     |
| Transactions   | ACID-compliant              | BASE (eventual consistency)      |
| Best For       | Structured, relational data | Unstructured, hierarchical data  |

**One-liner:** "SQL = rigid spreadsheets with JOINS. MongoDB = flexible JSON files in folders (collections)."

## Mongoose Schema + Model

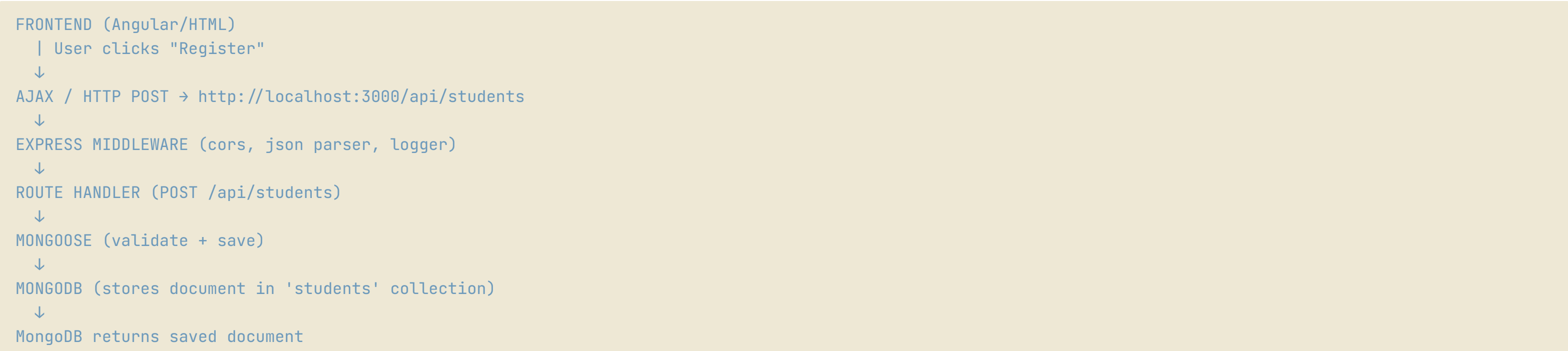
A Mongoose Schema defines the structure of the documents within a collection, including field types, defaults, and validation rules. A Model is a compiled version of the schema used to query the database.

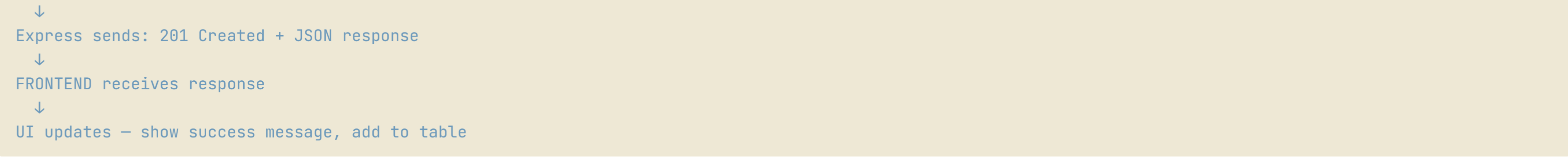
## Complete CRUD Routes

CRUD operations map to HTTP methods:

- Create:** `POST` request using `model.save()` or `model.create()`.
- Read:** `GET` request using `model.find()` or `model.findById()`.
- Update:** `PUT` / `PATCH` request using `model.findByIdAndUpdate()`.
- Delete:** `DELETE` request using `model.findByIdAndDelete()`.

## Full CRUD Flow Diagram (Viva Answer)





**Viva Quote:** "Our frontend sends a POST request with student JSON. Express receives it, the middleware parses the body, the route handler creates a Mongoose model instance, calls `.save()`, and MongoDB stores it. We respond with status 201 and the saved document. The Angular component's `.subscribe()` receives the response and updates the table."

## SECTION 4 — AUTHENTICATION

### Session vs JWT Authentication

| Feature     | Session Auth                  | JWT Auth                         |
|-------------|-------------------------------|----------------------------------|
| Storage     | Server-side session store     | Client-side token (localStorage) |
| Stateful?   | Stateful (server remembers)   | Stateless (server forgets)       |
| Scalability | Hard (server sticky sessions) | Easy (any server can verify)     |
| Logout      | Delete session from server    | Token expires or blacklisted     |
| Used In     | Traditional MVC apps          | REST APIs, SPAs                  |

### Basic HTTP Auth in Express

Basic HTTP Authentication involves checking the `Authorization` header, decoding the base64 credentials, and verifying them against stored records before allowing access to a route.

## VIVA Q&A — Unit 4

#### Q1: What is Node.js? Why use it for backend?

Node.js is a JavaScript runtime built on V8 engine. Use it for backend because: same language (JS) for frontend + backend, non-blocking I/O handles many concurrent requests efficiently, huge NPM ecosystem, great for real-time applications.

#### Q2: What is middleware in Express?

Middleware is a function with `(req, res, next)` parameters that runs between receiving a request and sending a response. Uses: logging, authentication, body parsing, error handling. Must call `next()` to pass control to the next middleware, or send a response to end the chain.

#### Q3: What is REST API? Explain CRUD with HTTP methods.

REST API uses HTTP methods to perform operations: **GET** = Read, **POST** = Create, **PUT** = Update, **DELETE** = Remove. It's stateless — each request is independent. URLs represent resources: `/api/students` = all students, `/api/students/:id` = specific student.

#### Q4: What is Mongoose? Why use it with MongoDB?

Mongoose is an ODM (Object Data Modeling) library for MongoDB and Node.js. It provides: Schema definition (structure for documents), built-in validation, middleware hooks (pre/post save), and a clean API for CRUD operations. MongoDB is schema-less, but Mongoose adds structure and safety.

#### Q5: What is the difference between `find()` and `findById()` ?

`find(filter)` returns an array of documents matching the filter — used to get all/many. `findById(id)` returns a single document by its MongoDB `_id` — used to get one specific record.

### ⚠ Oral Trap Questions

#### Trap: "What is the `_id` field in MongoDB?"

MongoDB automatically adds `_id` to every document — it's a unique `ObjectId` (12-byte BSON type). It's automatically indexed. Never set `_id` manually unless you have a specific reason.

#### Trap: "What happens if you don't call `next()` in middleware?"

The request hangs forever — the client receives no response and eventually times out. Always call `next()` to pass control, or send a `res.json()` / `res.send()` to end the request-response cycle.

## 📋 Assignment 3 — Practical Viva Flow

"I created a Node.js + Express REST API with 4 endpoints for CRUD operations on student data. MongoDB is used as the database with Mongoose ODM for schema validation. The 4 APIs are: **POST /api/students** (create), **GET /api/students** (read all), **PUT /api/students/:id** (update), **DELETE /api/students/:id** (delete). For static file hosting, I used `express.static('public')` middleware."

## 🔥 High Probability Questions

1. What is Node.js? How is it different from other server-side languages?
2. Explain non-blocking I/O with Event Loop
3. What is Express.js? Why use it?
4. Explain middleware with example
5. What is REST API? HTTP methods for CRUD?
6. SQL vs NoSQL — when to use which?
7. What is Mongoose? Schema and Model?
8. Write a POST route in Express that saves to MongoDB
9. What is `async/await` in Node.js?

10. What is the full CRUD flow (frontend to database)?